Experiment 3: Matrix-Vector and Matrix-Matrix Multiplication using MapReduce

MapReduce Laboratory

Aryan Awasthi
Roll No.: 102315182
Subgroup: 402

1. Aim

To write and implement MapReduce programs in Python for (i) matrix-vector multiplication and (ii) matrix-matrix multiplication.

2. Theory

Storing a matrix for MapReduce. A large matrix is stored as one line per non-zero element, name,row,column,value, e.g. M,0,3,3 means M[0][3] = 3. Zero elements are left out, so sparse matrices take little space. Each line is processed independently, which suits the map phase. Indices start at 0.

(i) Matrix-vector multiplication x = M v

For an m x n matrix M and a vector v of length n, each element of the result is

x_i = sum over j of m_ij x v_j

The vector is usually small enough to fit in memory, so it is sent to every mapper as a side file through the distributed cache (-files vector_v.txt).

Phase	Work
Map	For each element m_ij, emit i TAB m_ij x v_j.
Shuffle and Sort	All partial products of row i reach the same reducer.
Reduce	Add the partial products of row i to get x_i.

Addition is associative and commutative, so the reducer also works as a combiner.

(ii) Matrix-matrix multiplication C = A B

For A (m x n) and B (n x p), each result cell is

C[i][k] = sum over j of a_ij x b_jk

The program uses the **one-pass algorithm**. The key is the result cell (i, k). Each element is sent to every cell that needs it: a_ij is needed by the whole row i of C (all p columns), and b_jk by the whole column k of C (all m rows).

Phase	Work
Map	For a_ij emit (i,k) TAB A,j,a_ij for k = 0 .. p-1. For b_jk emit (i,k) TAB B,j,b_jk for i = 0 .. m-1.
Shuffle and Sort	All A and B values needed for cell (i, k) reach one reducer.
Reduce	Match the A and B values on j, multiply each pair and add the products to get C[i][k].

The mapper must know m and p, so the driver passes them with -cmdenv ROWS_A=m -cmdenv COLS_B=p. For very large matrices a two-pass algorithm can be used instead: the first job joins A and B on j and emits the products, and the second job adds them per (i, k).

3. Algorithm

(i) Matrix-vector

1. Each mapper loads v from the side file into a dictionary.
2. For each line M,i,j,m, emit i TAB m x v[j].
3. The reducer adds all values of the same i and emits i TAB x_i.

(ii) Matrix-matrix

1. For each line A,i,j,a, emit i,k TAB A,j,a for every column k of B.
2. For each line B,j,k,b, emit i,k TAB B,j,b for every row i of A.
3. For each key i,k, the reducer stores the A values and the B values by j. It then adds a_j x b_j over every j present in both, and emits i,k TAB C[i][k].

4. Program

mv_mapper.py

```
#!/usr/bin/env python3
"""(i) Matrix-vector multiplication x = M v, mapper.
In : M,i,j,m_ij lines of the matrix
Side input: the vector v, shipped to every mapper with -files vector_v.txt (lines v,j,v_j).
        The path can be changed with the VECTOR_FILE environment variable.
Out: i \t m_ij * v_j
"""
import os
import sys

v = {}
with open(os.environ.get("VECTOR_FILE", "vector_v.txt")) as f:
    for line in f:
        _, j, value = line.strip().split(",")
        v[int(j)] = float(value)

for line in sys.stdin:
    parts = line.strip().split(",")
    if len(parts) != 4 or parts[0] != "M":
        continue
    _, i, j, m = parts
    print(f"{i}\t{float(m) * v.get(int(j), 0.0):g}")
```

mv_reducer.py (also the combiner)

```
#!/usr/bin/env python3
"""(i) Matrix-vector multiplication, reducer (also the combiner).
In : i \t partial product   (sorted by i)
Out: i \t x_i = sum over j of m_ij * v_j
"""
import sys
```

```
    current, total = None, 0.0
    for line in sys.stdin:
        i, value = line.rstrip("\n").split("\t", 1)
        if i != current:
            if current is not None:
                print(f"{current}\t{total:g}")
            current, total = i, 0.0
        total += float(value)
    if current is not None:
        print(f"{current}\t{total:g}")
```

mm_mapper.py

```
    #!/usr/bin/env python3
    """(ii) Matrix-matrix multiplication C = A B (one-pass algorithm), mapper.
    A is m x n, B is n x p. The driver passes m and p: -cmdenv ROWS_A=m -cmdenv COLS_B=p
    In : A,i,j,a_ij  and  B,j,k,b_jk  lines
    Out: every element of A and B is sent to each result cell C[i][k] that needs it:
        A,i,j,a  ->  (i,k) \t A,j,a   for k = 0 .. p-1
        B,j,k,b  ->  (i,k) \t B,j,b   for i = 0 .. m-1
    """
    import os
    import sys

    m = int(os.environ["ROWS_A"])
    p = int(os.environ["COLS_B"])

    for line in sys.stdin:
        parts = line.strip().split(",")
        if len(parts) != 4:
            continue
        name, r, c, value = parts
        if name == "A":
            for k in range(p):
                print(f"{r},{k}\tA,{c},{value}")
        elif name == "B":
            for i in range(m):
                print(f"{i},{c}\tB,{r},{value}")
```

mm_reducer.py

```
    #!/usr/bin/env python3
    """(ii) Matrix-matrix multiplication, reducer.
    In : i,k \t A,j,a  or  B,j,b   (sorted by i,k)
    Out: i,k \t C[i][k] = sum over j of a_ij * b_jk
    Values are matched on j; a j present in only one matrix is a zero and adds nothing.
    """
    import sys
```

```
def emit(key, a, b):
    total = sum(a[j] * b[j] for j in a if j in b)
    print(f"{key}\t{total:g}")


current, a, b = None, {}, {}
for line in sys.stdin:
    key, value = line.rstrip("\n").split("\t", 1)
    name, j, x = value.split(",")
    if key != current:
        if current is not None:
            emit(current, a, b)
        current, a, b = key, {}, {}
    (a if name == "A" else b)[int(j)] = float(x)
if current is not None:
    emit(current, a, b)
```

5. Execution

Locally (Windows, macOS or Linux, Python 3 only):

```
$ python run_demo.py
```

The equivalent Unix pipelines:

```
$ VECTOR_FILE=data/vector_v.txt python3 mv_mapper.py < data/matrix_M.txt | sort -n |
python3 mv_reducer.py
$ cat data/matrix_A.txt data/matrix_B.txt | ROWS_A=3 COLS_B=2 python3
mm_mapper.py | sort | python3 mm_reducer.py
```

On Hadoop (run_hadoop.sh):

```
$ hadoop jar $STREAMING_JAR -files mv_mapper.py,mv_reducer.py,data/vector_v.txt \
    -mapper "python3 mv_mapper.py" -combiner "python3 mv_reducer.py" \
    -reducer "python3 mv_reducer.py" -input /user/$USER/matrix/mv_in -output
/user/$USER/matrix/mv_out
$ hadoop jar $STREAMING_JAR -files mm_mapper.py,mm_reducer.py -cmdenv
ROWS_A=3 -cmdenv COLS_B=2 \
    -mapper "python3 mm_mapper.py" -reducer "python3 mm_reducer.py" \
    -input /user/$USER/matrix/mm_in -output /user/$USER/matrix/mm_out
```

6. Output

The output below is the real output of python run_demo.py, showing the data after every phase.

Part (i): Matrix-vector multiplication x = M v

Matrix M (3 x 4)

	col 0	col 1	col 2	col 3
row 0	2	0	1	3
row 1	1	4	0	2
row 2	0	3	5	1

Vector v (4 x 1)

	col 0
row 0	1
row 1	2
row 2	3
row 3	4

Input file data/matrix_M.txt (M,i,j,value; zeros omitted) (9 lines)

```
$ cat data/matrix_M.txt
M,0,0,2
M,0,2,1
M,0,3,3
M,1,0,1
M,1,1,4
M,1,3,2
M,2,1,3
M,2,2,5
M,2,3,1
```

Side file data/vector_v.txt (shipped to every mapper) (4 lines)

```
$ cat data/vector_v.txt
v,0,1
v,1,2
v,2,3
v,3,4
```

Mapper output (i TAB m_ij x v_j) (9 lines)

```
$ python mv_mapper.py < data/matrix_M.txt
0    2
0    3
0    12
1    1
1    8
1    8
2    6
2    15
2    4
```

Shuffle and Sort (grouped by row i) (9 lines)

```
$ ... | sort -n
0	2
0	3
0	12
1	1
1	8
1	8
2	6
2	15
2	4
```

Reducer output (i TAB x_i) (3 lines)

```
$ ... | sort -n | python mv_reducer.py
0	17
1	17
2	25
```

Result x = M v (3 x 1)

	col 0
row 0	17
row 1	17
row 2	25

Part (ii): Matrix-matrix multiplication C = A B

Matrix A (3 x 3)

	col 0	**col 1**	**col 2**
row 0	1	2	0
row 1	3	1	4
row 2	2	0	5

Matrix B (3 x 2)

	col 0	**col 1**
row 0	2	1
row 1	0	3
row 2	4	2

Input files data/matrix_A.txt and data/matrix_B.txt (12 lines)

```
$ cat data/matrix_A.txt data/matrix_B.txt
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,1
```

```
A,1,2,4
A,2,0,2
A,2,2,5
B,0,0,2
B,0,1,1
B,1,1,3
B,2,0,4
B,2,1,2
```

Mapper output (i,k TAB matrix,j,value) (29 lines)

```
$ ROWS_A=3 COLS_B=2 python mm_mapper.py < A and B
0,0    A,0,1
0,1    A,0,1
0,0    A,1,2
0,1    A,1,2
1,0    A,0,3
1,1    A,0,3
1,0    A,1,1
1,1    A,1,1
1,0    A,2,4
1,1    A,2,4
... (19 more lines)
```

Shuffle and Sort: the group for result cell C[0][0] (4 lines)

```
$ ... | sort | grep '^0,0'
0,0    A,0,1
0,0    A,1,2
0,0    B,0,2
0,0    B,2,4
```

Shuffle and Sort (all groups) (29 lines)

```
$ ... | sort
0,0    A,0,1
0,0    A,1,2
0,0    B,0,2
0,0    B,2,4
0,1    A,0,1
0,1    A,1,2
0,1    B,0,1
0,1    B,1,3
0,1    B,2,2
1,0    A,0,3
... (19 more lines)
```

Reducer output (i,k TAB C[i][k]) (6 lines)

```
$ ... | sort | python mm_reducer.py
0,0    2
0,1    7
1,0    22
1,1    14
2,0    24
2,1    12
```

Result C = A B (3 x 2)

	col 0	**col 1**
row 0	2	7
row 1	22	14
row 2	24	12

Summary and verification (4 lines)

```
$ python run_demo.py
C[0][0] = sum over j of A[0][j] x B[j][0] = 1 x 2 + 2 x 0 + 0 x 4 = 2
Matrix-vector: 9 non-zero entries of M -> 3 result rows
Matrix-matrix: 12 input entries -> 29 mapper records -> 6 result cells
Both results match plain Python multiplication: YES
```

7. Result

Both programs were implemented in Python with the MapReduce model and run successfully.

- **(i)** For the 3 x 4 matrix M and the vector v = (1, 2, 3, 4), the result is x = (17, 17, 25). The mapper produced one partial product for each of the 9 non-zero entries, and the reducer added them row by row.
- **(ii)** For the 3 x 3 matrix A and the 3 x 2 matrix B, the result is C = [[2, 7], [22, 14], [24, 12]]. The mapper copied the 12 input entries into 29 records, one for each result cell that needs them. Each of the 6 reducer calls joined the A and B values on j and added the products.

Both results match a direct multiplication in plain Python.